Milestone 3 — Model Evaluation & Performance Assessment

Project: AI_PrognosAI — Remaining Useful Life (RUL) Prediction

Date: October 24, 2025

Author: Manju Shree

1. Objective
Rigorously evaluate the trained model's performance using defined metrics and analyze its predictive accuracy. Deliverables include RMSE for the test set, comparison plots of predicted vs actual RUL, bias/error analysis, and a detailed evaluation report.

2. Summary of Implementation & Scripts
The evaluation pipeline includes:
- Loading preprocessed test sequences (npz files) and the trained model (HDF5).
- Making predictions on test data and computing RMSE, MAE, R^2 and an approximate accuracy measure.
- Saving results to CSV (results/model_performance.csv or results/model_performance_fd{fd}.csv).
- Generating three plots saved under ./graphs/: predicted_vs_actual, residual_distribution, residual_trend.
- Performing bias and error analysis (mean & std of residuals), classifying bias type and strength, and saving the analysis to results/model_bias_analysis.csv.

Example files produced:
- graphs/fd1_predicted_vs_actual.png
- graphs/fd1_residual_distribution.png
- graphs/fd1_residual_trend.png
- results/model_performance.csv
- results/model_bias_analysis.csv

3. Metrics & Interpretation
Key metrics computed:
- RMSE (Root Mean Squared Error): primary error magnitude metric.
- MAE (Mean Absolute Error): average absolute error.
- R^2 Score: proportion of variance explained by the model.
- Approx. Accuracy: 100 - (RMSE / max(actual) * 100) $\rightarrow$ rough estimate; interpret cautiously.

Interpretation guidance:
- Lower RMSE/MAE indicates better per-sample RUL estimates.
- R^2 close to 1 indicates strong fit; negative R^2 indicates the model is worse than predicting the mean.
- Residual plots and distribution identify bias, heteroscedasticity, or outliers.

4. Observations & Common Issues
- If residuals skew positive: model tends to under-predict (predicts lower RUL than actual), risking unexpected failures.
- If residuals skew negative: model tends to over-predict, causing delayed maintenance actions.
- Large variance in residuals suggests inconsistent model performance across life stages; consider segmenting predictions by lifecycle buckets.

Recommendations:
- Use group-based evaluation per engine to measure per-engine performance.
- Consider percent-error thresholds and custom scoring (competition-style) if required.
- Visualize sample trajectories (actual vs predicted over sequence for a single engine) to inspect temporal behavior.

5. Example Commands to Run Evaluation
python evaluate_milestone3.py
(Ensure: processed npz files exist and model path points to the trained model.)

Milestone 4 — Risk Thresholding & Alert System

Project: AI_PrognosAI — Remaining Useful Life (RUL) Prediction

Date: October 24, 2025

Author: Manju Shree

1. Objective
Define and implement a mechanism to translate RUL predictions into actionable maintenance alerts. Deliverables include RUL thresholds for alert levels, logic for generating alerts, and examples of triggered alerts.

2. Summary of Implementation & Scripts
The risk/alert pipeline (milestone4_full.py) includes:
- Loading validation predictions and computing predicted RUL for each sample.
- Defining threshold levels, e.g.:
- Warning: predicted RUL <= 50 cycles
- Critical: predicted RUL <= 20 cycles
- Generating alerts and saving them to results/alerts_fd{fd}.csv
- Saving a final model (optimized_fd{fd}_milestone4.h5) and scaler (scaler_fd{fd}_milestone4.save)

Example outputs:
- results/alerts_fd1_milestone4.csv
- results/model_performance_fd1_milestone4.csv
- models_m2/optimized_fd1_milestone4.h5
- models_m2/scaler_fd1_milestone4.save

3. Threshold Design & Evaluation
Design principles:
- Thresholds should be set based on operational lead time requirements and historical failure data.
- Trade-offs:
- Lower thresholds produce fewer false positives but risk missing early failures.
- Higher thresholds produce earlier warnings but more false positives (maintenance cost).
- Recommendation: tune thresholds using historical labeled data and compute precision/recall for each alert level.

Suggested next steps:
- Evaluate thresholds by computing true positives / false positives using historical test episodes.
- Implement smoothing or aggregation (e.g., require N consecutive low-RUL predictions before triggering an alert) to reduce noise.
- Add severity scoring that accounts for prediction confidence (e.g., model ensembles or MC dropout).

4. Sample Alert Format
CSV columns produced:
- Sample_Index, Predicted_RUL, Alert_Level
Optional columns:
- Engine_ID, Cycle, Confidence, Timestamp

5. Deployment Notes
- Export model to TensorFlow SavedModel for production deployments.
- Integrate alert system into a lightweight API that receives telemetry, scales features (use saved scaler), runs model.predict and applies alert logic.
- Store alerts to a database and optionally send notifications (email/SMS) based on criticality.

6. How to Reproduce (Run Instructions)
1. Ensure model and processed NPZ files exist (processed/fd{fd}_test_ws{window}.npz).
2. Run evaluation script for Milestone 3.
3. Run milestone4_full.py to generate alerts.
4. Inspect results/ and graphs/ directories.

7. Appendix — Files to Check Into Repo
- evaluate_milestone3.py
- milestone4_full.py

- models_m2/*.h5
- graphs/*
- results/*

End of Milestone 3 & 4 Report.